

Tema 4.3.2: JavaScript Avanzado, Programación Funcional y Asincronía

Resumen

Este tema aborda las características avanzadas de **JavaScript (ES6+)** indispensables para el desarrollo de aplicaciones web complejas y escalables. Se analiza la programación funcional aplicada sobre arreglos (`map`, `filter`, `reduce`), las técnicas modernas de sintaxis (desestructuración, operadores *Spread* y *Rest*), la arquitectura del bucle de eventos del motor V8 (*Event Loop*, *Microtask Queue*), la gestión asíncrona no bloqueante mediante Promesas (`async/await`) y la Programación Orientada a Objetos basada en clases con módulos ES6.

Concepto Principal (El 'Qué')

1. Métodos Funcionales e Inmutables de Arreglos

JavaScript moderno desaconseja el uso de bucles tradicionales `for` o `while` para transformar colecciones de datos, reemplazándolos por métodos inmutables de orden superior:

- **`map(fn)`**: Transforma cada elemento de una lista mediante una función y retorna un **nuevo arreglo** de la misma longitud.
- **`filter(fn)`**: Evalúa un predicado booleano sobre cada elemento y retorna un **nuevo arreglo** conteniendo únicamente los elementos que cumplen la condición.
- **`reduce(fn, init)`**: Acumula y reduce los elementos de una colección a un **único valor** resultante (un número, un objeto o un mapa).
- **`find(fn)` / `some(fn)` / `every(fn)`**: Métodos de búsqueda corta que retornan el primer elemento coincidente o validaciones booleanas de colección.

2. Desestructuración y Operadores Spread / Rest (...)

- **Desestructuración (*Destructuring*):** Extrae propiedades de objetos o posiciones de arreglos directamente en variables individuales con una sintaxis limpia.
- **Operador Spread (...):** Expande los elementos de un iterable (arreglo u objeto) creando copias superficiales inmutables.
- **Operador Rest (...):** Agrupa múltiples argumentos o propiedades sobrantes dentro de una única variable de tipo arreglo.

```
const usuario = { id: 101, nombre: 'Ana', rol: 'ADMIN', pais: 'CR' };
// Desestructuración con renombrado y valores por defecto
const { nombre, rol: perfilUsuario, estado = 'ACTIVO' } = usuario;

// Copia inmutable con Spread y sobreescritura de propiedades
const usuarioActualizado = { ...usuario, rol: 'SUPERADMIN' };
```

3. El Bucle de Eventos de V8 (*Event Loop*) y la Asincronía

JavaScript ejecuta código de forma monohilo (*single-threaded*). Para procesar operaciones E/S lentas (como peticiones HTTP a servidores o temporizadores) sin congelar la interfaz, el motor V8 utiliza el **Event Loop**:

1. **Call Stack (Pila de Invocación):** Almacena las funciones en ejecución síncrona.
2. **Web APIs:** Delegación de tareas asíncronas lentas (temporizadores, `fetch`, eventos DOM) ejecutadas fuera de la pila principal.
3. **Microtask Queue (Pila de Microtarear):** Almacena las respuestas de **Promesas** (`then/catch/await`). Tiene prioridad absoluta sobre las tareas ordinarias.
4. **Task Queue / Macrotask Queue:** Almacena callbacks ordinarios (`setTimeout`, `setInterval`).

4. Promesas y Sintaxis `async/await`

Una **Promesa** es un objeto que representa el resultado eventual (éxito o falla) de una operación asíncrona. Posee tres estados: Pending, Fulfilled o Rejected.

La sintaxis `async/await` es azúcar sintáctico construido sobre Promesas que permite escribir código asíncrono no bloqueante con la legibilidad del código síncrono secuencial.

Analogía (La Intuición)

Imagine la cocina de un restaurante de alta cocina atendido por un chef estrella:

- **Call Stack (El Chef en la Estufa):** El chef solo tiene dos manos (un único hilo de ejecución) y puede picar una cebolla a la vez. Si el chef tuviera que quedarse estático esperando 45 minutos a que el horno asara un pavo, el restaurante colapsaría.
- **Web APIs y Event Loop (El Horno y los Temporizadores):** El chef programa el temporizador del horno (delegación de tarea asíncrona a la Web API) y libera sus manos de inmediato para seguir picando vegetales en el *Call Stack*. Cuando el horno suena (promesa resuelta), coloca la notificación en la bandeja de entrada (*Microtask Queue*).
- **Event Loop (El Asistente de Cocina):** El asistente observa continuamente si el chef terminó de picar la cebolla actual (*Call Stack vacío*). Tan pronto como el chef se libera, el asistente le entrega la bandeja del pavo asado (*Microtask*) para que la emplate sin perder un segundo.

Ejemplo de Código e Implementación (El 'Cómo')

El siguiente código demuestra el uso avanzado de clases ES6+, desestructuración, métodos de arreglos y procesamiento asíncrono paralelo con `Promise.all`:

```
// 1. Programación Orientada a Objetos en ES6 con campos privados (#)
export class ServicioEnvios {
  #apiKey; // Campo privado de clase (ES2022)

  constructor(apiKey) {
    this.#apiKey = apiKey;
  }

  async obtenerEnvioPorId(id) {
    const respuesta = await fetch(`/api/v1/envios/${id}`, {
      headers: { 'Authorization': `Bearer ${this.#apiKey}` }
    });
    if (!respuesta.ok) throw new Error(`Envío #${id} no encontrado`);
    return await respuesta.json();
  }
}
```

```

    }
  }

  // Método que procesa múltiples peticiones asíncronas en paralelo
  async obtenerEnviosParalelos(listaIds) {
    const promesas = listaIds.map(id => this.obtenerEnvioPorId(id));
    // Espera a que todas las promesas se resuelvan concurrentemente
    return await Promise.all(promesas);
  }
}

// 2. Programación Funcional y Transformación de Datos
export const analizarEstadisticas = (envios) => {
  // Filtrado e inmutabilidad
  const enviosEntregados = envios.filter(e => e.estado === 'ENTREGADO');

  // Mapeo y desestructuración
  const resmenes = enviosEntregados.map(({ id, pesoKg, cliente }) => ({
    codigo: `ENV-${id}`,
    peso: pesoKg,
    nombreCliente: cliente?.nombre ?? 'Desconocido' // Optional Chaining & Nullish Coalescing
  }));

  // Reducción acumulativa
  const pesoTotalKg = resmenes.reduce((acc, curr) => acc + curr.peso, 0);

  return { resmenes, pesoTotalKg, total: resmenes.length };
};

```

Explicación Paso a Paso del Código:

1. **Campos Privados (#apiKey):** Característica de ES2022 que encapsula variables privadas dentro de la clase sin exponerlas en las instancias públicas.
2. **Promise.all(promesas):** Ejecuta múltiples peticiones HTTP en paralelo en lugar de realizarlas de forma secuencial síncrona en un bucle `for`, reduciendo el tiempo total de respuesta al tiempo de la petición más lenta.
3. **filter, map, reduce:** Encadena transformaciones de datos inmutables sobre el arreglo de envíos sin modificar el arreglo original.
4. **?. y ?? (Optional Chaining & Nullish Coalescing):** `cliente?.nombre` evita errores runtime `TypeError: Cannot read properties of undefined` si el objeto `cliente` es nulo, mientras que `??` asigna un valor de respaldo sólo si la expresión evaluada es `null` o `undefined`.

Trampas Comunes

Ejecutar Peticiones Asíncronas Secuenciales en Bucles `for...of` con `await`:

- *Error:* Hacer `for (const id of ids) { await fetch(id); }`, sumando los tiempos de latencia de cada petición en serie.
- *Solución:* Disparar todas las peticiones simultáneamente con `map()` y esperar el arreglo de promesas mediante `Promise.all()`.

Mutar el Arreglo Original dentro de Métodos Funcionales:

- *Error:* Invocar `sort()` o `splice()` dentro de un pipeline funcional, modificando de forma colateral el estado original de los datos.
- *Solución:* Hacer una copia previa mediante el operador Spread (`[...arreglo].sort()`) o utilizar los nuevos métodos inmutables de ES2023 (`toSorted()`, `toSpliced()`).

Olvidar Retornar Valores en `map()` o `reduce()`:

- *Error:* Escribir un bloque `map(({ id }) => { id * 2; })` con llaves `{}` sin la palabra clave `return`, produciendo un arreglo lleno de elementos `undefined`.
- *Solución:* Retornar explícitamente el valor o utilizar la sintaxis abreviada de retorno implícito `map(({ id }) => id * 2)`.

Pregunta de Reflexión

En el procesamiento de eventos asíncronos en el cliente:

¿Por qué una Promesa resuelta colocada en la *Microtask Queue* se ejecuta con prioridad absoluta por el *Event Loop* antes de que el motor V8 procese cualquier temporizador `setTimeout` ubicado en la *Macrotask Queue*, y qué impacto tiene este orden en la fluidez de renderizado del navegador?